FlashAttention for Scalable Vector Architectures

Sonia Rani Gupta
Chalmers University of Technology
Gothenburg, Sweden
soniar@chalmers.se

Nikela Papadopoulou
University of Glasgow
Glasgow, UK
nikela.papadopoulou@glasgow.ac.uk

Miquel Pericàs
Chalmers University of Technology
Gothenburg, Sweden
miquelp@chalmers.se

Abstract

Inference with transformer models on CPUs is increasingly important, especially for Small Language Models (SLMs), where vector architectures are emerging as a promising execution substrate. The attention module is a major bottleneck due to high memory bandwidth requirements; FlashAttention mitigates this by fusing operations to improve data locality and reduce intermediate memory traffic. In this paper, we present FlashAttention-V, a blocked FlashAttention for scalable vector architectures that adapts efficiently from short to very long vectors by exploiting parallelism across attention heads, inter-head packing to enable efficient utilization of vector lengths beyond the head dimension, and improving vector register utilization and memory access locality. We integrate FlashAttention-V into ggml within `llama.cpp` and evaluate it on TinyLlama, Llama 3.2, Qwen2.5, and Pythia-410M using `gem5` and a Banana Pi BPI-F3. On the Banana Pi BPI-F3, we confirm that loop reordering and loop unrolling across attention heads are effective optimization principles, scaling performance gains with larger models and most pronounced with short contexts and during decoding. Simulation-based analysis shows that FlashAttention-V achieves 22×-42× speedup over scalar FlashAttention at 512-bit VL in prefill, with an additional 2×-2.5× gain scaling to 64 lanes and 4096-bit VL. During decode, FlashAttention-V achieves 8×-11× speedup using 512-bit vector lengths over scalar FlashAttention, with performance showing diminishing sensitivity to vector width and lane count due to single-token, memory-bound execution. We further identify structural bottlenecks in Q8_0 quantized linear layers that limit arithmetic amortization under long-vector execution, consistent across RVV and Arm SVE, indicating that current quantization formats pose a fundamental challenge to long-vector scalability.

1 Introduction

Transformer-based models underpin modern AI systems, powering applications ranging from chatbots and coding assistants to machine translation and AI agents, spanning a wide range of scales. Large Language Models (LLMs), with hundreds of billions of parameters, are typically deployed in data centers, whereas Small Language Models (SLMs), in the 1-10B parameter range, are optimized for edge deployment [19, 23, 46]. Across this spectrum, transformer inference demands high throughput, low latency, and strong energy efficiency, from large-scale server deployments to battery-constrained edge platforms [3, 4].

The attention module is a core component of all transformer architectures [41]. Multi-head attention (MHA) extends the original formulation by computing multiple attention heads in parallel, enabling the model to capture diverse relational patterns. To reduce the memory and bandwidth overhead of storing the key-value (KV) cache during inference, variants such as Multi-Query Attention (MQA) and Grouped-Query Attention (GQA) share keys and values across query heads, reducing the KV cache footprint [1, 23, 37].

Efficient execution of these attention variants has been explored across diverse hardware platforms. Attention implementations have been optimized for GPUs [15, 20, 32, 42] and dedicated AI accelerators [17, 18, 42] to maximize throughput for both training and inference. They have also been explored on CPUs to improve transformers inference due to the CPUs' high availability, low latency, and portability across edge devices and data centers [15, 40, 42].

Although these variants differ in how queries, keys, and values are organized, they all rely on the same underlying self-attention computation. Self-attention requires computing and storing large intermediate matrices, such as score matrix $S = QK^T$, where Q and K represent Queries and Keys, as well as the resulting softmax attention weights. This leads to significant memory capacity and bandwidth overhead, which becomes particularly pronounced as the sequence length grows. To address this memory bottleneck, Rabe and Staats [34] introduced a memory-efficient attention algorithm with online softmax, enabling exact self-attention without storing the full intermediate score matrix S. Building on this foundation, FlashAttention [11, 12] further optimizes execution by using tiling and fused kernels to keep intermediate data in on-chip GPU memory. While originally optimized for GPUs, recent work has explored FlashAttention on CPUs with vector units to improve transformer inference [15, 39].

Modern vector processors with vector-length-agnostic Instruction Set Architectures (ISAs), such as the RISC-V Vector Extension (RVV) and the ARM Scalable Vector Extension (Arm SVE), support variable vector lengths. This makes them attractive for HPC and AI workloads, which can exploit the SIMD parallelism exposed by these architectures. However, FlashAttention exposes limited vector-level parallelism along the head dimension. Queries, Keys, and Values have dimensions N×D, where N is the sequence length, and D is the head dimension. The FlashAttention output has the same shape, $N \times D$, with the size of N varying widely (collapsing to 1 during token generation) and D being small, typically 64 or 128 (see Table 3). Existing implementations of FlashAttention on RISC-V, ARM, and x86 follow a restrictive design choice: the vector length (VL) is tied to D. The vectorized FlashAttention implementations in llama.cpp are all limited to $VL \leq D$ [15]. Titopoulos et al. [39] vectorize FlashAttention on RVV specifically for $VL \leq D$. MEATTEN [13], a NEON-based fused-attention implementation, vectorizes over the head dimension d_k, thereby imposing the same restriction. For 16-bit precision, these implementations are able to utilize vector lengths only up to 1024–2048 bits, while more aggressively quantized models utilize even less. This limits the scalability potential of FlashAttention on long vector architectures, where vector lengths exceed the head dimension, leading to underutilization of Single Instruction, Multiple Data (SIMD) resources.

In this work, we bridge this gap by optimizing FlashAttention for vector architectures across a wide range of vector lengths, enabling efficient utilization of both conventional and long vector registers. We propose *FlashAttention-V*, a redesigned FlashAttention algorithm that exploits parallelism across attention heads. By reordering loops, applying loop unrolling, and applying inter-head packing, FlashAttention-V maps multiple heads into a single vector register, enabling the efficient utilization of vector lengths exceeding the head dimension D. Loop unrolling further exposes instruction-level parallelism, allowing FlashAttention-V to efficiently utilize vector registers and scale across both conventional and long vector lengths in prefill and decode stages. Our design natively supports MHA and GQA, avoiding redundant memory accesses for shared key-value.

We implement FlashAttention-V in the ggml framework [14] within llama.cpp [15], a framework commonly used as a research platform [22, 30]. We use vector intrinsics of the RISC-V ISA [35] and the Arm SVE ISA[5] to vectorize our kernels. We evaluate FlashAttention-V on decoder-only Large Language Models (LLMs) that operate in two stages: prefill (processes input tokens and computes the KV cache) and decode (generates one token at a time). We use three GQA models: TinyLlama [44], Llama 3.2 [27], and Qwen2.5 [33], as well as Pythia-410M [7], to evaluate MHA. We validate the functional correctness of FlashAttention-V with Arm SVE using QEMU to demonstrate portability across vector ISAs. We evaluate FlashAttention-V with RVV on a Banana Pi BPI-F3 RISC-V board and use gem5 [8] for RVV to assess scalability up to 8192-bit vector lengths. To reflect real hardware behavior, where a fixed number of vector lanes processing longer vector lengths increases per-instruction execution latency, we extend gem5's MinorCPU with vector length-proportional SIMD functional unit latencies, informed by the lane-based microarchitecture of Vitruvius+ [28]. Finally, we extend our analysis beyond the attention mechanism to quantify bottlenecks in quantized linear projection and feed-forward layers, showing that vector packing and masked reduction overheads significantly limit scalability with longer VLs.

The contributions of this paper are as follows:

- We propose FlashAttention-V, a FlashAttention algorithm for vector architectures that exploits parallelism across attention heads and introduces inter-head packing, enabling effective utilization of vector lengths beyond the head dimension ($VL > D$). Combined with loop reordering, inter-head unrolling, maximized vector register reuse, and native MHA/GQA support, FlashAttention-V achieves 22×–42× speedup over scalar (non-vectorized) FlashAttention in prefill and 8×–11× in decode at 512-bit vector lengths using gem5 on RVV. Validation on Banana Pi BPI-F3 confirms these gains with 12×-14× over scalar FlashAttention.
- We evaluate the scalability of FlashAttention-V using `gem5`, establishing upper-bound speedups of 2×-3× in prefill and 1.2× in decode across vector lengths up to 8192 bits. Using a latency-aware per-lane model, we show that configurations with up to 64 vector lanes and 4096-bit vectors retain strong scalability, while highlighting the vector-length overheads that limit performance beyond 4096-bit vectors. These findings provide actionable insights for hardware and vector-length-aware FlashAttention optimization on long-vector architectures, which have not been quantified previously.
- We characterize the structural incompatibility between Q8_0 quantization and long-vector execution in linear projection and feed-forward modules. Through microbenchmark analysis on RISC-V, we identify packing and masked reduction as the dominant bottlenecks, consuming 60% of execution cycles at 2048-bit VL. Validation on Arm SVE at 2048-bit VL confirms that these overheads are architecture-agnostic, stemming from the interleaved weight-scale layout of Q8_0, which introduces VL-dependent costs that prevent arithmetic amortization. This highlights a fundamental limitation for long-vector execution in quantized models.

The rest of this paper is organized as follows. Section 2 provides background on transformer models. Section 3 views related work. Section 4 describes the algorithmic transformations of FlashAttention and self-attention, and analyzes linear projection and feed-forward layers under Q8_0 quantization. Section 5 presents the experimental platforms and setup. Sections 6 and 7 evaluate our FlashAttention-V and analyze structural incompatibilities between quantized models and long-vector execution. We conclude the paper in Section 8.

2 Background

In this paper, we consider decoder-only models that use attention mechanisms such as Multi-Head Attention (MHA) or its efficient variant, Grouped-Query Attention (GQA), which reduces key–value cache overhead by sharing keys and values across query heads. These models are built upon the same underlying self-attention mechanism. ***Self-attention*** lets each token attend to all others [41]. Input embeddings are projected into Queries (Q), Keys (K), and Values (V), and attention is computed via scaled dot product and softmax: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{D}}\right)V$ where D is the head dimension. Multi-head attention applies this in parallel across multiple heads to capture diverse relationships. ***FlashAttention*** is an optimized implementation of self-attention that reduces memory usage and improves computational efficiency [12]. Instead of materializing the full attention matrix in memory, it tiles the computation across blocks, maintaining running maxima and normalization factors for numerically stable on-the-fly softmax, avoiding intermediate memory overhead.

`llama.cpp` [15] is an open-source CPU inference framework built on `ggml` [14], a tensor library supporting quantized operations and vectorized instructions across AVX, Arm NEON, Arm SVE, and RVV. In `llama.cpp`, the attention module is implemented using both self-attention and FlashAttention.

3 Related Work

Several works have focused on optimizing attention mechanisms. We first discuss algorithmic improvements to self-attention, followed by FlashAttention variants. Rabe and Staats [34] introduce a memory-efficient self-attention algorithm that avoids $O(n^2)$ memory usage. The FlashAttention algorithm [12] follows similar principles: it is an optimized implementation of the self-attention to

reduce the memory bandwidth. It was further optimized by Dao et al. [11], who proposed FlashAttention-2.

We next discuss CPU-specific implementations of self-attention and FlashAttention. Zhang et al. [45] propose NIOT, a framework for efficient inference using kernel fusion and tiling, evaluated on BERT and ViT on Xeon 6226R with AVX-512. XNNPACK [16] is a hand-tuned library for ARM and x86 CPUs that provides a fused SDPA operator with thread-level parallelism. Fu et al. [13] propose MEATTEN, a memory-efficient attention scheme for ARM NEON-based multi-core CPUs. Martínez et al. [25] optimize self-attention on ARM and RISC-V CPUs for encoder-only models such as BERT. Rodrigo et al. [30] optimize LLM inference in `llama.cpp` for the many-core RISC-V Sophon SG2042 [9]. Garcia et al. [24] benchmark LLM inference on the same platform using PyTorch with OpenBLAS and BLIS backends. Liu et al. [22] optimize key kernels (vector dot product and layer normalization) in `llama.cpp` on the Banana Pi BPI-F3 RISC-V platform. On RISC-V vector architectures, Titopoulos et al. [39] propose a FlashAttention variant restricted to $VL \leq D$, evaluated on Gemma-2 and Qwen models using gem5. `llama.cpp` introduced a GEMM-based vectorized attention alongside a reduction-based implementation [15], both limited to $VL \leq D$. OneDNN exposes SDPA through a graph-level fusion API [40], requiring intermediate storage of Query–Key dot products. It also supports GQA [21] through a graph-level pipeline combining reshape, MatMul, scaling, masking, and softmax into a single kernel for x86 CPUs and Xe GPUs. However, this remains a high-level fusion approach targeting server-class systems, and does not address kernel-level vector optimization for RISC-V edge processors.

The existing work on FlashAttention for RVV has been explored only in scenarios where the vector length is less than or equal to the head dimension, leaving the potential of long vector configurations unexploited. In this work, we redesign FlashAttention to exploit long vector lengths by exploiting parallelism across multiple attention heads, with native GQA support to eliminate redundant K/V loads.

4 Algorithmic Optimizations

In this section, we first present the algorithmic optimizations in FlashAttention-V, followed by optimizations in the self-attention implementation and an analysis of the performance characteristics of the quantized model.

4.1 FlashAttention-V Optimizations

For each attention head, the Query (Q), Key (K), and Value (V) tensors have shape (N × D), where N is the sequence length and D is the head dimension. In the decode stage, this reduces to (1 × D). The output matrix has the same dimensions, N×D and 1×D, per attention head for the prefill and decode stages, respectively. The `ggml` library includes an initial implementation for FlashAttention based on the streaming softmax formulation of [34]. A high-level pseudo-code of the ggml vectorized (`ggml-vec`) implementation of FlashAttention is presented in Algorithm 1. The kernels *kq_vec_dot*, *ggml_vec_scale*, and *ggml_vec_mad* (lines 6, 17,26, and 35) vectorize over the head dimension (D) and are limited by its size. Since these loops operate over D, the maximum exploitable vector length is bounded by D elements. For the common case of $D = 64$ or 128 with `FP16`, this corresponds to 1024-bit or 2048-bit VLs. Any vector length beyond these thresholds introduces no additional parallelism within a single head, as there are insufficient elements to fill those vector registers. Therefore, this FlashAttention implementation saturates vector utilization once the vector length exceeds D, as additional vector width cannot be effectively exploited within a single attention head. Our preliminary experimental evaluation validates this observation. To address this limitation, we redesign the ggml FlashAttention implementation and introduce FlashAttention-V, which exploits parallelism across attention heads to utilize longer vector registers. For scalable vector architectures, FlashAttention-V supports two execution regimes based on the hardware vector length relative to the head dimension: $VL > D$ (enabling inter-head packing) and $VL \leq D$ (no inter-head packing). Across these regimes, FlashAttention-V combines inter-head packing (for $VL > D$) with cache- and register-aware optimizations, blocking, loop reordering, and loop unrolling over attention heads, and GQA-aware data reuse, to improve data locality, vector utilization, and instruction-level parallelism.

Algorithm 1 `ggml-vec` FlashAttention in llama.cpp

1: Loop $j_1 \leftarrow 0$ **to** $N * attention\_heads$ **step** 1 //N - sequence length
2: check for causal masking
3: $M = -INFINITY$ //Keep global maximum
4: $Sum = 0.0$ //keep global sum
5: Loop $ic \leftarrow 0$ **to** N **step** 1
6: **for** $i \leftarrow 0$, **to** D **do** //$kq\_vec\_dot(D, \&s, K_{ptr}, Q_{ptr})$
7: $avl \leftarrow setvl(D - i)$
8: $V\_Q = Q_{ptr}[0 : avl]$ $V\_K = K_{ptr}[0 : avl]$
9: $s = vfmv(vfredusum(fmul(V\_Q, V\_K)))$
10: **end for**
11: $S \leftarrow s * scale$
12: $M_{old} \leftarrow M$ //Keep the old maximum in M_{old}
13: $ms \leftarrow 1.0$ $vs \leftarrow 1.0$
14: **if** $S > M$ **then** //check for new maximum
15: $M = S$
16: $ms = expf(M_{old} - M)$ //Calculate rescaling factor
17: **for** $i \leftarrow 0$, **to** D **do** ////$ggml\_vec\_scale(D, VKQ, ms)$
18: $avl \leftarrow setvl(D - i)$
19: $VKQ = VKQ\_array[0 : avl]$
20: $VKQ = fmul(VKQ, ms, avl)$ //VKQ rescale
21: $VKQ\_array[0 : avl] = VKQ$
22: **end for**
23: **else**
24: $vs = expf(S - M)$ //Compute softmax weight
25: **end if**
26: **for** $i \leftarrow 0$, **to** D **do** //$ggml\_vec\_mad(D, VKQ, V_{ptr}, vs)$
27: $avl \leftarrow setvl(D - i)$
28: $VKQ = VKQ\_array[0 : avl]$ $V\_V = V_{ptr}[0 : avl]$
29: $VKQ = vfmadd(VKQ, V\_V, vs, avl)$ //accumulate VKQ
30: $VKQ\_array[0 : avl] = VKQ$
31: **end for**
32: $Sum = Sum * ms + vs$ //Online softmax sum
33: End Loop ic
34: $Sum_{inv} = 1/Sum$
35: $ggml\_vec\_scale(D, VKQ, Sum_{inv})$//normalize VKQ
36: $dst = VKQ$ ▷ put result back

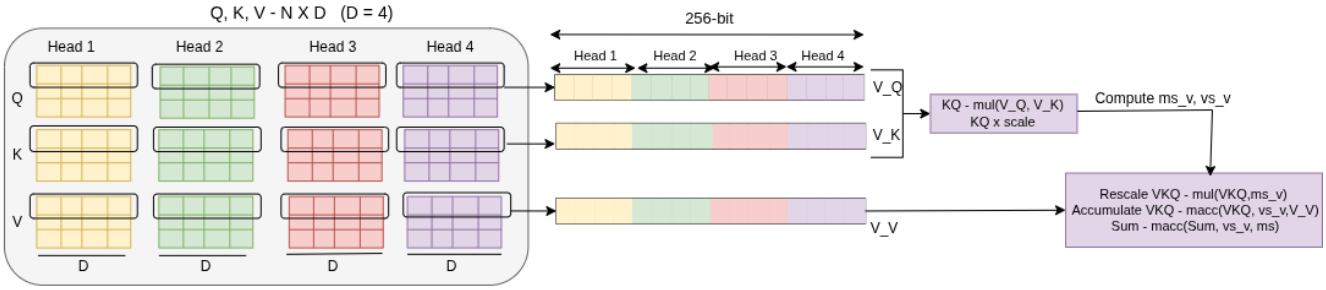

Figure 1: Inter-head packing into a single vector register to improve vector utilization across attention heads with FP16

Algorithm 2 FlashAttention-V for VL > D

1: Initialize: $VL \leftarrow$ vsetmaxvl(), $avl \leftarrow D$ $itr \leftarrow VL/D$
2: Loop j_1 and k_1 to N steps $block\_N$ and $block\_k$ //N - sequence length
3: Loop $j \leftarrow 0$ **to** $block\_N$
4: **for** $i_1 \leftarrow 0$ **to** $attention\_heads$ **step** U **do** //U (unroll factor)
5: **for** $i \leftarrow 0$ **to** R **do** //R(register_utilization_factor) blocks of itr
6: $V\_Q[i] \leftarrow Q_{ptr}[i_1, i * itr : VL]$
7: **end for**
8: **if** $k_1 = 0$ **then**
9: $Sum_v[0{:}R] = 0, VM[0{:}R] = -\infty, VKQ[0{:}R] = 0$
10: **else**
11: Load from previous $S_{array}, M_{array}, VKQ\_array$
12: **end if**
13: **for** $ic \leftarrow k_1$ **to** $k_1 + block\_K$ **do**
14: Check for Causal masking
15: **for** $i \leftarrow 0$ **to** R **do**
16: **if** Grouped-Query Attention **then**
17: Load $K_{ptr}[ic, i * itr : avl]$ and Pack in V_K //avl=D
18: **else**
19: $V\_K[i] \leftarrow vle(K_{ptr}[ic, i * itr : VL])$ //load full VL
20: **end if**
21: $KQ[i] \leftarrow vfmul(V\_Q[i], V\_K[i])$ //KQ_vec_dot
22: $KQ[i] \leftarrow vfmul(KQ[i], scale)$ //Scale score
23: $VS[i] = vfredusum_m(mask, KQ[i])$ //scores
24: $VM_{old}[i] = VM[i]$ //keep old maximum
25: $VM = vmerge(VM, VS, mask)$ //new maximum
26: $ms\_v[i] = exp\_v(VM_{old}[i], VM[i])$ //rescale factor
27: $vs\_v[i] = exp\_v(VS[i], VM[i])$ //softmax weights
28: Load V_V and accumulate similar to K
 //VKQ rescale
29: $VKQ[i] \leftarrow vfmul\_m(mask, VKQ[i], ms\_v[i])$
 //accumulate VKQ
30: $VKQ[i] \leftarrow vfmacc(VKQ[i], V\_V[i], vs\_v[i])$
 //Online softmax sum
31: $Sum_v[i] \leftarrow vfmacc(vs\_v[i], Sum_v[i], ms\_v[i])$
32: **end for**
33: **end for**
34: Update $S_{array}, M_{array}, VKQ\_array$
35: Normalize VKQ and store in dst
36: **end for**

4.1.1 FlashAttention-V optimizations when $VL > D$. Given that vector parallelism over D saturates at $VL = D$, individual attention heads cannot fully utilize vector registers when $VL > D$. We therefore exploit parallelism across attention heads to fill longer vector lengths. In this regime, FlashAttention-V additionally introduces inter-head packing, mapping multiple attention heads into a single vector register to improve vector utilization under wide hardware configurations. We further improve efficiency by retaining intermediate results in vector registers across iterations, reducing memory traffic and improving data locality. We implement blocked FlashAttention based on the non-vectorized `ggml` baseline, with blocking strategies inspired by FlashAttention-2 [11], using cache-aware block sizes. A key observation is that attention heads are independent, as each head computes its own query (Q), key (K), and value (V) projections without cross-head dependencies. We exploit this independence as an additional source of parallelism by processing multiple heads simultaneously. Analysis of the `ggml` memory layout shows that Q, K, and V tensors across heads are stored contiguously. However, the baseline loop order does not exploit this structure. By reordering loops over attention heads, FlashAttention-V improves spatial locality and enables more unit-stride-friendly memory accesses, allowing multiple heads to be mapped efficiently into a single vector register. As illustrated in Figure 1, four FP16 elements across four heads can be packed into a single vector register(256-bits), fully utilizing vector capacity. To further improve utilization, we apply loop unrolling over attention heads, which increases vector register occupancy and exposes instruction-level parallelism (ILP) by keeping multiple vector registers active concurrently. Together, loop reordering and unrolling improve performance along two dimensions: (i) increasing effective vector width through inter-head packing, and (ii) improving instruction-level parallelism and memory latency hiding.

Algorithm 2 shows the pseudo code for FlashAttention-V for RVV when the vector length is greater than D. Line 1 initializes three key parameters: VL is set to the maximum available vector length, avl sets the vector length to D (limit operations to D), and $itr = VL \div D$ determines how many attention heads fit within a vector register. The outer loops in line 2 tile the computation over seq_length in blocks of $block\_N$ and $block\_K$, selected to reduce memory traffic and improve data reuse. Loop over $attention\_heads$ is reordered to line 4 and incremented by a loop unroll factor U, to expose contiguous heads that enable inter-head packing and improve vector register utilization. Lines 5–7 load Q into vector registers to maximize register reuse within the innermost loop and reduce memory bandwidth pressure. Lines 8-12 are used to keep S_{array}, M_{array}, and VKQ_array the global sum, global max, and

global VKQ to support the online softmax operation [34]. They loaded in vector registers named as sum_v, VM, and VKQ.

In the inner-most loop, Keys are packed in lines 16-20 based on whether the model uses GQA or MHA. In the MHA case, K and V are contiguously aligned in memory and loaded using the *vle* vector instruction in line 19. For grouped-query attention or multi-query attention, K and V values based on the *avl* are loaded and replicated in vector registers using the *vslideup* vector instruction. If the attention heads to KV head ratio is 4:1, i.e., four attention heads are shared among KV heads, then only a single load instruction is required to load K, after which the *vslideup* instruction is used to replicate it across the full vector register. A similar packing mechanism is applied when loading V. Lines 21–31 implement core attention computation in a fully vectorized manner. The dot product between V_Q and V_K is computed and scaled in lines 21-22. The running maximum is updated in lines 23-25, from which the rescaling factor ms_v and the softmax weight vs_v are derived using the vectorized exponential function $exp\_v()$ in lines 26-27. We use exp_v from SIXTE [36], modified to handle FP16 data precision, infinity values, and masking. The accumulated output VKQ is rescaled and updated using `vfmul` and `vfmacc` vector instructions in lines 29-30, and the running softmax sum sum_v is updated in line 31. After all tiles are processed, VKQ is normalized and written to dst, completing the attention computation.

Tunable parameters in Algorithm 2. The block sizes $block\_N$ and $block\_K$ are tuned to better fit within the L2 cache. In our implementation, U is decided based on the vector length, the precision in which elements are stored, the head dimension, and the number of attention heads. Since the number of $attention\_heads$ varies across models, we must also consider it when deciding on the loop-unrolling. We set the unrolling factor U as $U = \frac{VL}{D \cdot b} \times R$. Here, b is the element bit-width and R is the register utilization factor, which captures the additional loop unrolling enabled by available vector registers and the number of $attention\_heads$. For example, in TinyLlama and Llama 3.2, the head dimension is 64, so using $attention\_heads = 8$ maps cleanly onto an 8192-bit vector width (8 × 64 × 16 bits). In these models, the total number of heads is 32; therefore, after processing the first 8 heads, the remaining 24 heads allow increasing unrolling from 8 to 32 while still maintaining full utilization of the 8192-bit vectors, by setting R to 4.

4.1.2 FlashAttention-V optimizations when $VL \leq D$. In this regime, vector parallelism operates within individual attention heads. Unlike the $VL > D$ regime: no inter-head packing is applied, Q, K, and V are loaded according to the available vector length, and intermediate VKQ results are accumulated and written back to memory between iterations, consistent with `ggml-vec` (Algorithm 1). FlashAttention-V extends this with optimizations shared across both regimes: (i) cache-aware blocking tuned to the cache hierarchy, consistent with Algorithm 2; (ii) loop reordering over attention heads; (iii) loop unrolling by factor U on attention heads to enhance parallelism and maximize vector register utilization; and (iv) GQA support, where keys and values are loaded once and reused across shared attention heads to reduce memory bandwidth pressure.

4.2 Self-attention Optimizations

For the self-attention implementation, ggml provides a blocked implementation involved in llama.cpp to accelerate the transformers on CPUs. Drawing on the optimization principles of FlashAttention-V, we demonstrate that loop reordering and inter-head parallelism are broadly applicable to self-attention kernels, improving vector utilization across the QK^T, softmax, and QKV accumulation stages during prefill computation.

In the case of QK^T, we apply vectorization across the head dimension to exploit data-level parallelism and enable contiguous memory accesses. Tokens in a Query are stored contiguously, allowing multiple tokens from a single head to be packed into long vector registers. For the Keys, one row of D elements is loaded from memory and packed into long vector registers using the `vslideup` intrinsic, since computation is performed for one head at a time if $D < VL$. Multiple `vfmul_vv` instructions then multiply the packed Query vectors with the loaded Key row, enabling maximum reuse of the Queries. For example, at 8192-bit VL, 32 FP16 Query rows of dimension $D = 64$(as in TinyLlama) are packed across 4 vector registers per attention head, each multiplied against a single Key row, maximizing Key reuse within the innermost loop, amortizing the memory load cost of K across multiple token computations.

In softmax, masking is applied to the attention score matrix, followed by the softmax operation. The score matrix has dimensions $N \times N$, where N is the sequence length, allowing efficient use of long vector lengths. We reorder the attention-head loop and apply a loop unrolling factor of 16, balancing register usage and avoiding spilling. Since tokens across heads share the same mask, loaded data can be reused effectively. For QK^TV, dimensions for the calculated attention weight matrix and Value matrix are N×N and N×D, respectively, for the prefill stage. Here, N is large enough to utilize the long VLs. We apply a loop unrolling factor of 16, chosen to balance register utilization and avoid spilling, over the score matrix rows, multiplying 16 rows of the score matrix against a single Value row to produce 16 output results simultaneously, maximizing Value reuse within the innermost loop. Increasing loop unrolling from 16 to 32 degrades performance due to register spilling.

4.3 Linear Projection and Feed-Forward

Both prefill and decode consist of an input embedding, multiple decoder blocks, an output embedding, and a sampling layer. Each block includes Q/K/V projections, attention, and a feed-forward module. We evaluate TinyLlama on a Banana Pi BPI-F3 using a non-vectorized llama.cpp compiled with `LLVM` compiler. Llama-bench profiling shows FlashAttention dominates prefill (50% per layer) but contributes only (10%) in decode, where linear projections and feed-forward layers dominate. Therefore, in addition to FlashAttention-V, we evaluate these modules for a complete decode-stage analysis.

Both modules are linear projections(input multiplied by a learned weight matrix). In decode, single-token inputs reduce them to General Matrix-Vector (GEMV) operations implemented via `vec_dot` in llama.cpp. We evaluate 8-bit GGUF Q8_0 quantization, where weights are stored in 32-element blocks with symmetric INT8 values and a per-block FP16 scale (no zero-point). This layout follows an Array of Structures (AoS) format, with 32 INT8 weights followed by a 2-byte scale. On RISC-V, this AoS layout limits long-vector

Table 1: Simulated system configuration

Component	Specification
Core frequency	2GHz
Vector length	512-bits to 8192-bits
Memory type	DDR3-1600
Memory bandwidth	12.8 GB/s per core
Cache hierarchy	L1: 64 kB; L2: 1 MB

execution because contiguous loads interleave weights and scales, reducing effective vector utilization. To fully utilize wider vector configurations, the scattered 8-bit weights must be reorganized into contiguous vector registers (packing), and partial results must be reduced at the block level (unpacking). We explore mechanisms for efficient packing and reduction to enable execution under longer VLs and we discuss the challenges under Section 7.

5 Methodology

5.1 Experimental Platform

We use gem5 [8] 24.0.0.1, a cycle-accurate simulator configured with the RISC-V in-order *RiscvMinorCPU* model with RVV v1.0 support. The simulated memory subsystem uses DDR3-1600 with a per-core bandwidth of 12.8 GB/s, within the same order of magnitude as the Intel Xeon Max 9480 with HBM (19 GB/s) [26], ensuring that the results reflect realistic bandwidth constraints. The CPU model includes two levels of cache, configured similarly to AMD Zen 4 processors [2]. The main parameters are summarized in Table 1.

We note that in gem5's MinorCPU model, the MinorDefaultFloatSimdFU (a floating-point and SIMD functional unit) executes all operations with a fixed latency of 6 cycles, independent of operation type or vector width. Since the out-of-order *RiscvO3CPU* model could not complete simulations due to memory exhaustion, we instead scale functional unit latencies in the *RiscvMinorCPU* model to better reflect real hardware behavior.

We scale SIMD FU latencies in the *RiscvMinorCPU* using $opLat = startup + \max\left(0, \left\lceil \frac{VL}{PhysicalWidth} \right\rceil - 1\right)$, where $PhysicalWidth$ represents the physical throughput per cycle of the vector unit to model realistic vector latency. We use 512 bits/cycle as the reference throughput, modeled as the aggregate throughput of Vitruvius+'s 8 lanes operating at 64 bits per lane per cycle [28]. We derive the base latencies from `O3_ARM_v7a.py` in the gem5 configurations, as summarized in Table 2. The model extends to wider designs: Ara2 with 16 lanes gives 1024 bits, and AraXL with up to 64 lanes gives 4096 bits [29, 31]. The startup term accounts for the FU pipeline depth at the base vector width; additional passes beyond the first each contribute one extra cycle.

To validate the correctness, we use QEMU 9.0.4 configured with `rv64,v=true,zvfh=true,zfhmin=true,vext_spec=v1.0`. We note that gem5 25.0.0.1 produced inaccurate results despite identical cycle counts for RVV; we therefore use gem5 24.0.0.1 throughout.[1] Finally, we evaluate FlashAttention-V on the Banana Pi BPI-F3, which supports RVV v1.0 with a 256-bit vector length. For the Arm SVE packing analysis in linear projection and feed-forward layers, we use gem5 25.0.0.1 configured with the same configuration as the RVV setup.

Table 2: Comparison of MinorCPU fixed latencies, and ARM OoO base latencies (O3_ARM_v7a.py), with VL-scaling indicated for each operation class.

Op Class	MinorCPU Fixed (cycles)	O3_ARM base (cycles)	VL-Scaled
SimdFloatAdd	6	5	Yes
SimdFloatAlu	6	5	Yes
SimdFloatMult	6	3	Yes
SimdFloatMultAcc	6	5	Yes
SimdFloatReduceAdd	6	4	Yes
FloatAdd	6	5	No
FloatCvt	6	5	No
FloatMult	6	4	No
FloatMultAcc	6	5	No

5.2 Experimental Setup

In this work, we evaluate FlashAttention-V in both the prefill and decode stages across four models: three Grouped-Query Attention (GQA) models, TinyLlama [44], Llama 3.2 [27], and Qwen2.5 [33], and one Multi-Head Attention (MHA) model, Pythia-410M [7]. Their dimensions are summarized in Table 3. Since RISC-V lacks BF16 support, our implementation uses FP32 and FP16 precision, consistent with recent studies [13, 25]. We apply Q4_K_M and Q8_0 (4-bit and 8-bit block quantization schemes, respectively) quantization in linear projection and feed-forward layers; by the time data reaches FlashAttention, tensors are already expanded to native FP16 or FP32 formats. All experiments use a batch size of 1, corresponding to a single inference request.

We use vector intrinsics from the RISC-V ISA [35] to vectorize the attention implementation in llama.cpp for RVV. We use the EPI fork of the LLVM Clang cross-compiler version 21.0.0 for RVV [10], with `-O3` and `-rv64gcv_zvfh` compiler flags to enable the support of half-precision with vector intrinsics to compile llama.cpp with our FlashAttention-V. We use GCC cross-compiler version 15.2.1 for Arm SVE, with `-O3` and `-march=armv8.2-a+sve2` compiler flags. Functional correctness of our FlashAttention-V on Arm SVE is verified using qemu-aarch64 version 8.2.2; performance evaluation is conducted on RVV only. We integrate our implementation with ggml in llama.cpp project[2] All models are evaluated using the `llama-bench` benchmarking tool from llama.cpp.

Baseline: We evaluate FlashAttention-V against three baselines. Here, `ggml-scalar` is the non-vectorized FlashAttention implementation provided by ggml in llama.cpp. `ggml-vec-fp16` is the vectorized FlashAttention implementation provided by ggml in llama.cpp. `ggml-vec-fp32` is derived from `ggml-vec-fp16` by explicitly setting the FP32 execution path for all vectorized functions. Titopoulos et al. [39] also provide a FlashAttention implementation for RVV; however, their tensor layout is incompatible with llama.cpp, precluding a direct comparison.

[1] Cycle counts remain identical between versions.

[2] https://github.com/ggml-org/llama.cpp, commit `5fc7cb2021231998f7fbeefbdf21feddf1b4d746`

Table 3: Dimensions of Decoder-only LLMs Used in Our Evaluation

Model Variants	Attention Heads	KV Heads	Hidden Size	Head Dimension (D)	Size
TinyLlama	32	4	2048	64	1.1B
Llama 3.2	32	8	2048	64	1B
Qwen2.5	16	2	2048	128	3B
Pythia	16	16	1024	64	410M

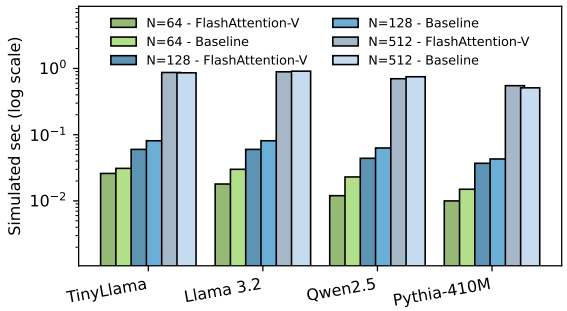

Figure 2: Prefill Performance Comparison of FlashAttention-V and `ggml-vec-fp16`

6 Evaluation of FlashAttention-V

In this section, we present the impact of our optimized FlashAttention on the Banana Pi BPI-F3. We evaluate both the performance of the attention module and end-to-end inference using TinyLlama, Llama-3.2, Qwen2.5, and Pythia-410M. We further use gem5@RVV to analyze FlashAttention-V with a 512-bit and larger VLs to assess its single-core scalability. In gem5, we simulate a single layer from both the prefill and decode stages. Since all layers operate on the same tensor dimensions, this is sufficient to capture behavior across the network while significantly reducing simulation time, especially in the prefill stage. When reporting FlashAttention performance, we report results for the attention module only; preprocessing and feed-forward modules are excluded from these measurements. All baselines are evaluated against llama.cpp [3].

6.1 FlashAttention-V on Banana Pi BPI-F3

We optimize FlashAttention-V with FP32 and FP16 implementations on RVV, which can efficiently utilize both shorter and longer vector lengths. On the Banana Pi platform, the vector length is 256 bits (i.e., $VL \leq D$), eliminating the need for packing across heads. However, we unroll the attention heads by a factor of loop unroll U as discussed in section 4. We unroll attention heads by a factor U, tuned over $\{1, 2, 4, 8\}$; performance degrades for $U > 4$, so we use $U = 4$ throughout. We use the TinyLlama 1.1B, Llama 3.2, Qwen2.5 models, and Pythia-410M for our analysis.

We first compare our FlashAttention-V (FP32) in the *prefill stage*. We compare against `ggml-scalar` and `ggml-vec-fp32` in the prefill stage. Our analysis shows that FlashAttention-V achieves 12× and 14× speedup over `ggml-scalar` for TinyLlama and Qwen2.5, respectively, and a 3.7× speedup over `ggml-vec-fp32` for both models. Compared to an optimized self-attention, FlashAttention-V achieves a 1.4× speedup for TinyLlama 1.1B, confirming the benefit of the FlashAttention algorithm.

Further, our analysis in Figure 2 shows that in the prefill stage, FlashAttention-V with FP16 matches `ggml-vec-fp16` at $N = 512$ and outperforms it at shorter sequence lengths ($N \leq 128$). For $N = 512$, FlashAttention-V performs comparably to `ggml-vec-fp16` on TinyLlama and Llama 3.2; on Qwen2.5, it achieves approximately 5% higher throughput, while on Pythia-410M it yields about 7% lower throughput at the same sequence length. This regression in Pythia-410M arises because it has fewer attention heads (16 vs 32) and uses MHA without key-value sharing, while Qwen2.5 benefits from a larger head dimension (128 vs 64), which provides more elements per head to amortize blocked computation and improve vector utilization.

We additionally evaluate different sequence lengths, namely $N \in \{64, 128, 256\}$, and FlashAttention-V achieves an average speedup of 1.2×–1.5× over `ggml-vec-fp16`, with the largest gains at $N = 64$ and $N = 128$ for TinyLlama, Llama 3.2, and Pythia-410M. For Qwen2.5, FlashAttention-V delivers 1.5×–2× speedup over `ggml-vec-fp16` at $N = 64$ and $N = 128$. Overall, FlashAttention-V is most effective at short sequence lengths ($N \leq 128$) conditions under which our blocked design maximizes data reuse and reduces memory pressure. These small token sizes align naturally with practical edge deployment scenarios; our experiments are conducted on the Banana Pi BPI-F3, a representative emerging IoT and embedded inference platform. On such resource-constrained devices, prefill sequence lengths of ≤128 tokens are common in practice and consistent with prior edge inference studies that adopt $N \leq 128$ as a baseline [6, 38, 43].

In the *decode stage* with both FP16 and FP32, our implementation achieves 4× - 5× speedups over `ggml-scalar` for the TinyLlama, Llama 3.2, Qwen2.5, and Pythia-410M models, respectively. Furthermore, FlashAttention-V with FP16 and FP32 delivers ~2× speedup over `ggml-vec-fp16` and `ggml-vec-fp32` for all models. Ultimately, this confirms that FlashAttention-V significantly accelerates both the prefill and decode stages, proving highly effective across all short-sequence regimes. We note that all optimizations discussed for FlashAttention-V are transferable to other vector architectures.

We next evaluate the *end-to-end performance* of the prefill and decode stages. For this, we integrate FlashAttention-V into `llama.cpp` [4]. In this version, `llama-simple` serves as a minimal inference application with FlashAttention enabled by default. We verify the correctness of FlashAttention-V by confirming that the generated tokens match those of the ggml baseline implementation. Using this setup, `llama-simple` achieves end-to-end throughput of 6 T/s and approximately 3 T/s for TinyLlama_Q4_K_M and TinyLlama_Q8_0, respectively, across both prefill and decode stages. Using `llama-bench` for the prefill stage (128 tokens and 8 threads), FlashAttention-V achieves a 4% improvement over the ggml vectorized baseline for Qwen2.5_Q4_K_M (3.52 T/s), while remaining comparable for TinyLlama_Q4_K_M (8.5 T/s) and Llama-3.2_Q4_K_M (6 T/s). We

[3] https://github.com/ggml-org/llama.cpp, commit `5fc7cb2021231998f7fbeefbdf21feddf1b4d746`

[4] https://github.com/ggml-org/llama.cpp, commit `d0061be838809230db7a4edf62bc9a098025ba98`

Table 4: Execution time for 1 FlashAttention using gem5@RVV with 512-bits VL for TinyLlama

Version	Prefill stage	Decode stage
Non-vectorized	21.38	0.000529
Vectorized by ggml with FP32	3.67	0.000300
FlashAttention-V with FP32	0.89	0.000077
FlashAttention-V with FP16	0.78	0.000070

note that the `llama-bench` evaluation for Qwen2.5_Q8_0 did not complete under the tested configuration. For the decode stage (128 generated tokens, 8 threads), FlashAttention-V achieves performance comparable to the baseline for Qwen2.5_Q4_K_M (2 T/s), TinyLlama_Q4_K_M (5.7 T/s), and Llama-3.2_Q4_K_M (5.3 T/s). These results are consistent with the overall system behavior: linear projection and feed-forward layers are identical across implementations, and the impact of FlashAttention is dominant in the prefill stage (approximately 50% of per-layer execution) but limited in decode (approximately 10%), leading to modest end-to-end gains.

6.2 FlashAttention-V using gem5

The ggml vectorized FlashAttention is limited to VL ≤ D and does not scale beyond the head dimension. To utilize longer vector lengths, FlashAttention-V employs parallelism across attention heads, using inter-head packing to map Q, K, V rows from multiple heads into a single vector register. We evaluate this using gem5 on RVV across vector lengths from 512 to 8192 bits.

We first tune the tunable parameters(loop unrolling factor and block size) of FlashAttention-V with a single prefill layer of TinyLlama at a sequence length $N = 512$ and a 512-bit vector length using gem5@RVV with the default in-order MinorCPU configuration. For algorithm $VL \leq D$, we achieve maximum utilization of the vector registers with unrolling factor $U = 4$. Increasing U to $U = 8$ leads to an 8% performance degradation at a 512-bit vector length. We then tune the block sizes *block_N* and *block_K*. Starting from sizes of 64×64 as the baseline, 128×128 achieves near-identical performance (0.97×), while 16×16 and 16×128 reduce performance to 0.50× and 0.89×, respectively. We therefore set $U = 4$ and $block\_N \times block\_K = 64 \times 64$ for all subsequent experiments.

We evaluate our FlashAttention-V against `ggml-scalar`, `ggml-vec-fp32`, and `ggml-vec-fp16` for a prefill and decode stage for TinyLlama. For the *prefill stage*, FlashAttention-V with FP32 achieves speedup of 24× and 4× compared to `ggml-scalar` and `ggml-vec-fp32`, respectively. With FP16, FlashAttention-V achieves a 28× speedup over `ggml-scalar`. For the *decode stage*, where the model generates one token at a time (making Q, K, V tensors of size $1 \times D$), achieving parallelism to fill the vector lengths is more challenging. In the decode stage, FlashAttention-V with FP32 achieves a 7.6× speedup over `ggml-scalar` and a 3.8× speedup over `ggml-vec-fp32` on TinyLlama. We note that we were unable to simulate `ggml-vec-fp16` with gem5 due to unsupported instructions.[5]

FP16 vs FP32. We compare FlashAttention-V with FP16 and FP32 at the smallest and largest VLs in our simulated environment. FP16 halves storage relative to FP32, allowing twice as many elements per vector register. We observe that FlashAttention-V with FP16 yields ~2×, over FlashAttention-V with FP32 with 8192-bit VL, but only 11% at 512-bit vectors. For a decode stage, we observe a ~1.5× speedup with 8192-bit vectors, while only a 7% improvement is seen with 512-bit vectors. For 512-bit vector lengths (VL<=D), algorithmic implementation uses the same unrolling factor in both FP32 and FP16, as no packing across multiple heads is required. FP16 enables performing arithmetic operations on twice the number of elements, but the number of attention scores computed remains unchanged. Consequently, the benefit of FP16 is limited by the softmax bottleneck rather than the arithmetic throughput.

[5]Specifically, gem5 does not support LMUL=8 for widening instructions when simulating `ggml-vec-fp16`.

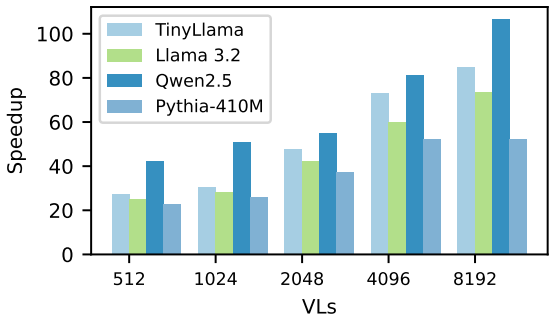

Figure 3: Prefill stage: Speedup across different vector lengths using gem5@RVV for TinyLlama, Llama 3.2, Qwen2.5, and Pythia-410M with 512 tokens, compared to `ggml-scalar`

Self-attention vs FlashAttention-V. We next evaluate the performance of FlashAttention-V over self-attention, after optimizing self-attention to utilize longer vector lengths, with FP16. We validate that our optimization renders self-attention scalable to longer vectors, yielding a speedup of 1.6× when scaling the vector length from 512 bits to 8192 bits. Our results show that FlashAttention-V gives a speedup of 1.3× and ~2.4× over the optimized self-attention algorithm with 512-bit and 8192-bit vector lengths, respectively, in the prefill stage, confirming the advantages of FlashAttention-V across vector lengths.

6.3 Scaling FlashAttention-Von different vector lengths

Further, we analyse the scalability with different vector lengths using gem5@RVV. This enables us to assess the effectiveness of FlashAttention-V across varying vector lengths. We note that we use FlashAttention-V with FP16 for our scalability analysis with gem5. We first simulate FlashAttention-V using the default operation latencies of the in-order CPU in gem5@RVV. We then evaluate performance using operation latencies scaled according to the number of vector lanes, as discussed in Section 5. For longer vector lengths ($VL > D$), we use a register utilization factor $R = 4$ to calculate the unrolling factor $U = \frac{VL}{D \cdot b} \times R$.

Prefill stage. Figure 3 shows that FlashAttention-V consistently improves performance over `ggml-scalar` across 512-bit to 8192-bit vector lengths on TinyLlama, Llama 3.2, Qwen2.5, and Pythia-410M under a gem5 in-order model configured with constant operation latencies to establish an upper bound for achievable speedup. At

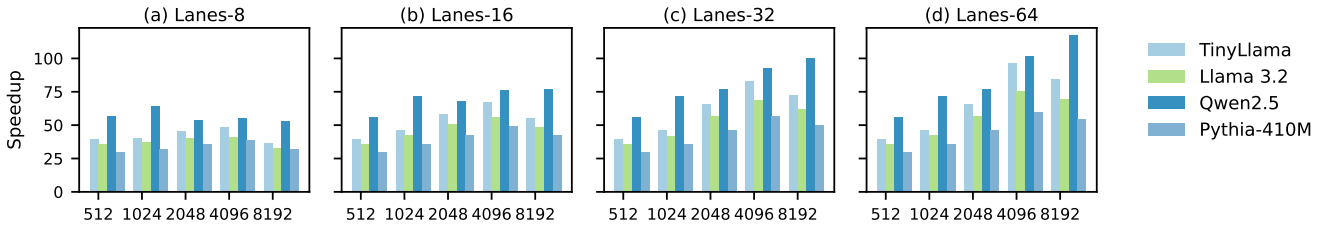

Figure 4: Prefill speedup across vector lengths (gem5@RVV, 512 tokens) for TinyLlama, Llama 3.2, Qwen2.5, and Pythia-410M, with VL-proportional operation latencies per vector lane, normalized to `ggml-scalar`.

the 512-bit vector length, FlashAttention-V achieves speedups of 22×–27× for TinyLlama, Llama 3.2, and Pythia-410M, and up to 42× for Qwen2.5. Increasing the vector length to 8192 bits yields an additional ∼3× speedup for TinyLlama and Llama 3.2, and ∼2.5× for Qwen2.5, demonstrating consistent benefits from wider vector utilization. The magnitude of these gains is governed by model architecture factors, such as per-head dimension and number of attention heads, which determine how effectively heads can be packed into vector registers. In particular, Qwen2.5 (head dimension 128) packs 4 heads per 8192-bit vector, whereas TinyLlama and Llama 3.2 (head dimension 64) pack 8 heads, resulting in better scalability. Pythia-410M, which is a smaller model, achieves its maximum speedup of 3× at 4096 bits, with full register utilization with $R = 4$ and $U = 16$. Beyond 4096 bits, the limited number of attention heads (16) does not allow the increase of the register utilization factor R, which is limited to $R = 2$, resulting in the same unrolling factor of $U = 16$. Overall, our results confirm that FlashAttention-V scales consistently with vector length across all models, with performance gains varying based on model architecture.

We next evaluate the scalability of FlashAttention-V with operation latencies scaled according to the vector length, by parameterizing the vector unit across 8, 16, 32, and 64 lanes. Figure 4 demonstrates that physical lane count is the primary determinant of performance scalability under realistic vector latency constraints. Lower lane counts exhibit early saturation: for the 8-lane configuration, performance degrades beyond 1024 bits, while 16- and 32-lane configurations extend this threshold to 4096 bits, with progressively improved scalability in intermediate ranges. In contrast, the 64-lane configuration achieves the best scalability, yielding 2×-2.5× speedups when increasing the vector length from 512 to 4096 bits, for all models, with up to an additional 1.15× gain for Qwen2.5 on 8192 bits. Increasing the lane count from 8 to 64 widens the physical throughput from 512 to 4096 bits per cycle, accommodating longer vectors in fewer passes and significantly reducing the total OpLat penalty. Overall, our results indicate that 64 vector lanes paired with a 4096-bit VL achieve better scalability, successfully balancing theoretical throughput gains against VL-dependent latency overheads.

Overall, under idealized fixed latencies, FlashAttention-V scales consistently up to 8192-bit vector lengths. However, we demonstrate that under realistic latency scaling, where performance is governed by VL-dependent latency overhead of the vector functional unit, configurations with 64 vector lanes and 4096-bit vectors achieve the highest scalability among the studied designs, sustaining 2×-2.5× speedup while avoiding the latency penalties that limit

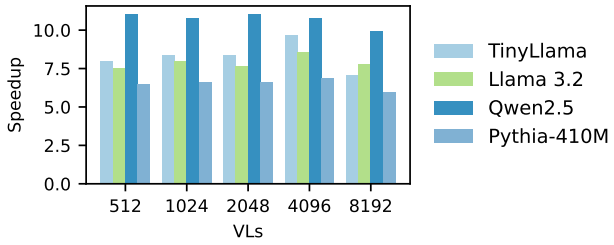

Figure 5: Decode stage: Speedup across vector lengths (VLs) on gem5@RVV for TinyLlama, Llama 3.2, Qwen 2.5, and Pythia-410M, relative to `ggml-scalar`

performance beyond 4096 bits. Our results also demonstrate that configurations with 64 vector lanes and 4096-bit vector lengths achieve comparable or higher speedups relative to the speedup achieved by constant-latency, indicating that FlashAttention-V is not limited by operation latency (OpLat) in the regime of long vector widths when considering 64 vector lanes.

Decode Stage. As discussed earlier, during the decode phase, only one token is generated at a time; thus, the Q, K, and V vectors for the new token have a dimension of $1 \times D$ per head. This inherent lack of parallelism makes it challenging to utilize longer vector lengths. Our FlashAttention-V implementation bypasses this constraint by distributing computation across attention heads, aligning decode-stage execution with the only available source of parallelism in single-token generation.

Figure 5 shows the performance of FlashAttention-V across vector lengths from 512-bit to 8192-bit on TinyLlama, Llama 3.2, Qwen2.5, and Pythia-410M over `ggml-scalar`, under a gem5 in-order configuration with constant latency across vector lengths to represent the upper bound of achievable speedup. At 512-bit vector length, FlashAttention-V achieves 8×–11× speedup across all models. Scaling to 4096 bits yields a modest ∼1.2× improvement for TinyLlama and Llama 3.2, while Pythia-410M and Qwen2.5 show no significant additional gains. This saturation arises because the decode stage operates in a sequential single-token regime, which limits inter-token reuse and limits the amount of parallel work available per step.When the vector registers become wider than a single attention head, the per-iteration workload is insufficient to amortize the overhead of packing/unpacking data into/from the wider vector registers, leading to diminishing returns from additional vector length scaling.

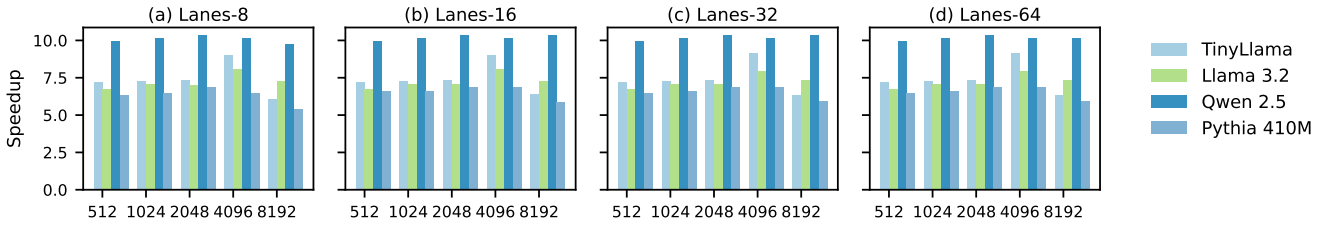

Figure 6: Decode stage: Speedup across different vector lengths (X-axis) using gem5@RVV for TinyLlama, Llama 3.2, Qwen2.5, and Pythia-410M, considering operation latencies per vector lane (baseline: `ggml-scalar`)

Figure 6 shows the speedup of FlashAttention-V across increasing vector lengths under scalable operation latencies, compared to `ggml-scalar` during the decode stage. Under a fixed-latency model, small but observable differences appear across vector lengths, indicating minor sensitivity to vector scaling. However, with realistic, scalable operation latencies, these differences are significantly reduced, and performance becomes nearly invariant across vector lengths and lane counts. This behavior again owes to the fact that the decode stage processes only a single token per step, leaving the total vectorizable work too small for operation latency differences across configurations to affect execution time. Consequently, decode performance is effectively insensitive to OpLat, with only small variation across vector lengths and lane configurations.

Overall, the optimizations applied in FlashAttention-V benefit both the prefill and decode stages. In the decode stage, where the sequence length is fixed at 1, parallelism can only be extracted across attention heads. Because the computational workload per iteration is small, the overhead of data packing and managing wider vector registers is not fully amortized. Even under these constraints, FlashAttention-V still achieves up to a 1.2× speedup with 4096-bit vector lengths for TinyLlama and Llama 3.2, demonstrating that its vector-level optimizations remain effective even in the most parallelism-constrained setting.

7 Structural Limits to Long-Vector Scaling in Quantized Layers

Vectorizing the `vec_dot` operation to exploit long vector lengths (VLs) in linear projection and feed-forward layers presents challenges distinct from attention kernels. In the ggml Q8_0 format, each quantized block contains 32 INT8 weights prefixed by a single FP16 scale, stored contiguously. This interleaved layout disrupts vectorization: vector loads over weights also fetch scale values, requiring explicit separation prior to computation. We take ggml's Q8_0 vectorized implementation as a baseline and evaluate three strategies to leverage longer VLs.

First, we apply register packing using `vslideup`, concatenating multiple quantized blocks into a single vector register. This is followed by widening multiply–accumulate and a masked reduction (`vwredsum_m`) to produce per-block results. To avoid iterative extraction via `vslidedown`, we rely entirely on masked reductions. Despite reducing arithmetic instructions, this approach degrades performance in TinyLlama by ~1.5×. To isolate the bottlenecks, we design a microbenchmark that replicates `vec_dot` at projection dimensions and evaluates packing and reduction independently. On gem5 with a 2048-bit VL, packing via `vslideup` accounts for 28% of cycles, masked reduction for 32%, and vector MAC for 40%. Thus, packing and reduction together consume 60% of execution time, exceeding arithmetic cost and negating the benefits of longer VLs. We also evaluate packing on Arm SVE using `svcreate4`, `svst4`, and predicate-based `svld1`; among these, `svld1` performs best (as in llama.cpp for 512-bit VL), yet still incurs ~ 20% overhead with gem5 with a 2048-bit VL.

Second, we restructure the computation to vectorize across the output dimension M, analogous to GEMV. This removes both packing and reduction overheads but introduces strided memory accesses (stride = 32) to respect quantization block boundaries. On the Banana Pi BPI-F3, this approach underperforms the baseline: non-unit strides degrade cache-line utilization, and increased memory latency outweighs the computational savings.

Third, we explore using LMUL=4 register groups with manual loop unrolling to map individual block loads into successive LMUL=1 segments, targeting up to 1024-bit effective VL. This avoids explicit packing entirely. However, the current `gem5` RISC-V vector model does not correctly simulate register-group type conversions when LMUL $\geq$ 4, precluding reliable cycle-count validation. We identify this as a gap in current simulation infrastructure and leave hardware validation for future work.

We conclude that, under Q8_0 quantization, linear projection and feed-forward modules do not benefit from long vector lengths in their current form. The limitation is structural: the interleaved scale–weight layout forces either non-unit-stride accesses or explicit packing, both of which introduce VL-dependent overheads that offset arithmetic gains. As quantized formats become the norm, the amortization argument for longer VLs weakens: when packing and reduction costs scale with VL alongside arithmetic, the overhead never gets amortized. Therefore, it becomes important to explore quantization formats or memory layouts that eliminate explicit packing overhead, would enable longer vector lengths to amortize arithmetic cost, and remain an open direction for future work.

8 Conclusion

This paper introduces FlashAttention-V, a blocked FlashAttention algorithm for scalable vector architectures that exploits parallelism across attention heads to utilize longer vector lengths and maximize register utilization and reuse. We implement and evaluate FlashAttention-V on RVV, and validate portability to Arm SVE via QEMU. Using gem5 on RVV at 512-bit VL, FlashAttention-V achieves 22×–42× speedup over non-vectorized FlashAttention in

prefill and 8×–11× in decode. Scalability analysis shows that 64 vector lanes paired with 4096-bit VL achieve optimal scalability, sustaining 2×–2.5× speedup in prefill and 1.2× in decode, beyond which VL-dependent latency penalties outweigh throughput gains. The relative speedup of 8192-bit over 512-bit VL remains stable as sequence length grows from 512 to 4096 tokens, confirming consistent scaling behavior. The core optimizations of FlashAttention-V, loop reordering, and inter-head unrolling are transferable across execution strategies, as confirmed by a 5% improvement when applied to the more recent tiled GEMM-based FlashAttention implementation [6] in `llama.cpp` on the Banana Pi BPI-F3. While FlashAttention-V demonstrates strong attention kernel performance, linear projection and feed-forward modules under Q8_0 quantization do not benefit from extended VLs due to structural layout constraints; exploring alternative quantization formats remains future work.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 4895–4901. doi:10.18653/v1/2023.emnlp-main.298

[2] AMD. [n. d.]. *AmdZen4.* https://www.amd.com/en/partner/articles/amd-ryzen-7000-series-desktop-processors.html

[3] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed- Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15. doi:10.1109/SC41404.2022.00051

[4] Mauricio Fadel Argerich and Marta Patiño-Martínez. 2024. Measuring and Improving the Energy Efficiency of Large Language Models Inference. *IEEE Access* 12 (2024), 80194–80207. doi:10.1109/ACCESS.2024.3409745

[5] Arm Limited. 2026. *Introducing SVE2.* Arm Developer. https://developer.arm.com/documentation/102340/0100/Introducing-SVE2 Accessed: April 30, 2026.

[6] Mayank Arya and Yogesh Simmhan. 2025. Understanding the Performance and Power of LLM Inferencing on Edge Accelerators. In *2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 1108–1111. doi:10.1109/IPDPSW66978.2025.00173

[7] Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. 2023. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. arXiv:2304.01373 [cs.CL] https://arxiv.org/abs/2304.01373

[8] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. 2011. The gem5 simulator. *SIGARCH Comput. Archit. News* 39, 2 (Aug. 2011), 1–7. doi:10.1145/2024716.2024718 GitHub repository: https://github.com/gem5/gem5.

[9] Nick Brown and Maurice Jamieson. 2024. Performance characterisation of the 64-core SG2042 RISC-V CPU for HPC. arXiv:2406.12394 [cs.DC] https://arxiv.org/abs/2406.12394

[10] BSC. 2023. *LLVM EPI Compiler.* https://ssh.hca.bsc.es/epi/ftp/

[11] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691 [cs.LG] https://arxiv.org/abs/2307.08691

[12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. arXiv:2205.14135 [cs.LG] https://arxiv.org/abs/2205.14135

[13] Xiao Fu, Weiling Yang, Dezun Dong, and Xing Su. 2024. Optimizing Attention by Exploiting Data Reuse on ARM Multi-core CPUs. In *Proceedings of the 38th ACM International Conference on Supercomputing* (Kyoto, Japan) *(ICS '24)*. Association for Computing Machinery, New York, NY, USA, 137–149. doi:10.1145/3650200.3656620

[14] Georgi Gerganov and contributors. 2022. *GGML: Tensor Library for Machine Learning.* https://github.com/ggml-org/llama.cpp/tree/master/ggml GitHub repository.

[15] Georgi Gerganov and contributors. 2023. *llama.cpp.* https://github.com/ggml-org/llama.cpp/blob/master/ggml/src/ggml-cpu/ops.cpp GitHub repository.

[16] Google. 2025. XNNPACK: High-Efficiency Floating-Point Neural Network Inference Operators. https://github.com/google/XNNPACK. Accessed: 2025.

[17] Google Cloud TPU Team. 2026. SplashAttention: TPU7x (Ironwood) performance optimizations. https://docs.cloud.google.com/tpu/docs/ironwood-performance. Accessed April 2026.

[18] Google JAX Team. 2025. JAX Pallas: Writing TPU kernels with Pallas. https://docs.jax.dev/en/latest/pallas/tpu/details.html. Accessed April 2026.

[19] Md Mahade Hasan, Muhammad Waseem, Kai-Kristian Kemell, Jussi Rasku, Juha Ala-Rantala, and Pekka Abrahamsson. 2026. Assessing Small Language Models for Code Generation: An Empirical Study with Benchmarks. arXiv:2507.03160 [cs.SE] https://arxiv.org/abs/2507.03160

[20] Hugging Face Transformers Team. 2026. Hugging Face Transformers. https://huggingface.co/docs/transformers/en/index. Accessed April 2026.

[21] Intel Corporation. 2025. Grouped Query Attention (GQA) — oneDNN v3.13.0 Documentation. https://uxlfoundation.github.io/oneDNN/dev_guide_graph_gqa.html. Accessed: 2025.

[22] Zhilong Liu, Long Peng, Wenzhu Wang, Ke Li, Binrui Zeng, Jie Yu, and Xiaodong Liu. 2025. Accelerating LLM Inference on RISC-V Edge Devices via Vector Extension Optimization. In *Advanced Intelligent Computing Technology and Applications*, De-Shuang Huang, Chuanlei Zhang, Qinhu Zhang, and Yijie Pan (Eds.). Springer Nature Singapore, Singapore, 515–526.

[23] Zhenyan Lu, Xiang Li, Dongqi Cai, Rongjie Yi, Fangming Liu, Xiwen Zhang, Nicholas D Lane, and Mengwei Xu. 2024. SMALL LANGUAGE MODELS: SURVEY, MEASUREMENTS, AND INSIGHTS. *arXiv preprint arXiv:2409.15790* (2024).

[24] Adriano Marques Garcia, Giulio Malenza, Robert Birke, and Marco Aldinucci. 2026. Inference performance of large language models on a 64-core RISC-V CPU with silicon-enabled vectors. *Future Generation Computer Systems* 177 (2026), 108242. doi:10.1016/j.future.2025.108242

[25] Héctor Martínez, Francisco D. Igual, Rafael Rodríguez-Sánchez, Sandra Catalán, Adrián Castelló, and Enrique S. Quintana-Ortí. 2024. Inference with Transformer Encoders on ARM and RISC-V Multicore Processors. In *Euro-Par 2024: Parallel Processing*, Jesus Carretero, Sameer Shende, Javier Garcia-Blas, Ivona Brandic, Katzalin Olcoz, and Martin Schreiber (Eds.). Springer Nature Switzerland, Cham, 377–392.

[26] John D. McCalpin. 2023. Bandwidth Limits in the Intel Xeon Max (Sapphire Rapids with HBM) Processors. In *High Performance Computing*, Amanda Bienz, Michèle Weiland, Marc Baboulin, and Carola Kruse (Eds.). Springer Nature Switzerland, Cham, 403–413.

[27] Meta AI. 2024. *Llama 3.* https://www.llama.com/models/llama-3/ Accessed: April 2026.

[28] Francesco Minervini, Oscar Palomar, Osman Unsal, Enrico Reggiani, Josue Quiroga, Joan Marimon, Carlos Rojas, Roger Figueras, Abraham Ruiz, Alberto Gonzalez, Jonnatan Mendoza, Ivan Vargas, César Hernandez, Joan Cabre, Lina Khoirunisya, Mustapha Bouhali, Julian Pavon, Francesc Moll, Mauro Olivieri, Mario Kovac, Mate Kovac, Leon Dragic, Mateo Valero, and Adrian Cristal. 2023. Vitruvius+: An Area-Efficient RISC-V Decoupled Vector Coprocessor for High Performance Computing Applications. *ACM Trans. Archit. Code Optim.* 20, 2, Article 28 (March 2023), 25 pages. doi:10.1145/3575861

[29] Matteo Perotti, Matheus Cavalcante, Renzo Andri, Lukas Cavigelli, and Luca Benini. 2024. Ara2: Exploring Single- and Multi-Core Vector Processing With an Efficient RVV 1.0 Compliant Open-Source Processor. *IEEE Trans. Comput.* 73, 7 (July 2024), 1822–1836. doi:10.1109/tc.2024.3388896

[30] Javier Jesus Poveda Rodrigo, Mohamed Amine Hamdi, Cyril Koenig, Alessio Burrello, Daniele Jahier Pagliari, and Luca Benini. 2025. POSTER: V-Seek: Optimizing LLM Reasoning on A Server-Class General-Purpose RISC-V Platform. In *Proceedings of the 22nd ACM International Conference on Computing Frontiers (CF '25)*. Association for Computing Machinery, New York, NY, USA, 224–225. doi:10.1145/3719276.3727954

[31] Navaneeth Kunhi Purayil, Matteo Perotti, Tim Fischer, and Luca Benini. 2025. AraXL: A Physically Scalable, Ultra-Wide RISC-V Vector Processor Design for Fast and Efficient Computation on Long Vectors. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–7. doi:10.23919/date64628.2025.10992880

[32] PyTorch Foundation. 2024. PyTorch 2.2: FlashAttention-v2 integration, AOTInductor. https://pytorch.org/blog/pytorch2-2/. Accessed: 2026-04-30.

[33] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2025. Qwen2.5 Technical Report. arXiv:2412.15115 [cs.CL] https://arxiv.org/abs/2412.15115

[34] Markus N. Rabe and Charles Staats. 2022. Self-attention Does Not Need $O(n^2)$ Memory. arXiv:2112.05682 [cs.LG] https://arxiv.org/abs/2112.05682

[6] https://github.com/ggml-org/llama.cpp, commit `5637536517ae4ed3eaa22b39c0d479e049097a9b`

[35] RISC-V Foundation. [n. d.]. *RISC-V "V" Vector Extension, Version 1.0.* https://github.com/riscv-non-isa/riscv-rvv-intrinsic-doc/blob/main/doc/rvv-intrinsic-spec.adoc

[36] SIXTE ORIOL LLENAS SEGURA. [n. d.]. *ACCELERATING DL WORKLOADS ON HPC ARCHITECTURES.* https://upcommons.upc.edu/server/api/core/bitstreams/0347b1aa-1a81-42df-b979-e035a65d0952/content

[37] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150 [cs.NE] https://arxiv.org/abs/1911.02150

[38] Chunlin Tian, Xinpeng Qin, Kahou Tam, Li Li, Zijian Wang, Yuanzhe Zhao, Minglei Zhang, and Chengzhong Xu. 2025. CLONE: customizing LLMs for efficient latency-aware inference at the edge. In *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference* (Boston, MA, USA) *(USENIX ATC '25)*. USENIX Association, USA, Article 34, 23 pages.

[39] Vasileios Titopoulos, Kosmas Alexandridis, and Giorgos Dimitrakopoulos. 2026. Vectorized FlashAttention with low-cost exponential computation in RISC-V vector processors: V. Titopoulos et al. *The Journal of Supercomputing* 82, 4 (2026), 189.

[40] UXL Foundation. 2025. Scaled Dot-Product Attention (SDPA) — oneDNN v3.13.0 Documentation. https://uxlfoundation.github.io/oneDNN/dev_guide_graph_sdpa.html. Accessed: April 2025.

[41] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[42] vLLM Team. 2026. vLLM: Easy, fast, and cheap LLM serving for everyone. https://docs.vllm.ai/en/stable/. Accessed April 2026. Tagline confirms production deployment across all major hardware.

[43] Run Wang, Gamze Islamoglu, Andrea Belano, Viviane Potocnik, Francesco Conti, Angelo Garofalo, and Luca Bonini. 2025. VEXP: A Low-Cost RISC-V ISA Extension for Accelerated Softmax Computation in Transformers . In *2025 IEEE 32nd Symposium on Computer Arithmetic (ARITH)*. IEEE Computer Society, Los Alamitos, CA, USA, 37–44. doi:10.1109/ARITH64983.2025.00016

[44] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. TinyLlama: An Open-Source Small Language Model. arXiv:2401.02385 [cs.CL] https://arxiv.org/abs/2401.02385

[45] Zining Zhang, Yao Chen, Bingsheng He, and Zhenjie Zhang. 2023. NIOT: A Novel Inference Optimization of Transformers on Modern CPUs. *IEEE Transactions on Parallel and Distributed Systems* 34, 6 (2023), 1982–1995. doi:10.1109/TPDS.2023.3269530

[46] Zeynep Örpek, Büşra Tural, and Zeynep Destan. 2024. The Language Model Revolution: LLM and SLM Analysis. In *2024 8th International Artificial Intelligence and Data Processing Symposium (IDAP)*. 1–4. doi:10.1109/IDAP64064.2024.10710677